



UNIVERSIDAD AUTÓNOMA DE QUERÉTARO

Facultad de Informática
Ingeniería en Software



SnitchAI

Documentación Técnica del Sistema

Plataforma de Generación Automática de Aplicaciones Móviles mediante Inteligencia Artificial

Proyecto Final

Integrante:

Arriola Aguillo Erick.319872
Arlin Estefany Montiel Ruiz Exp.319838
Gonzales Nieto Damian Elías Exp.319850
Rojas Piña Alan Salvador Exp.285490
Tienda Avila Sjany Exp.319877

Docente:

Axel Daniel Trejo González

Fecha de entrega: 20 de mayo de 2026

Índice de Contenidos

1. Introducción
2. Objetivos del Proyecto
3. Alcance del Sistema
4. Descripción General del Problema
5. Requerimientos del Sistema
6. Requerimientos Funcionales
7. Requerimientos No Funcionales
8. Arquitectura del Sistema
9. Tecnologías Utilizadas
10. Diseño del Sistema
11. Estructura del Proyecto
12. Descripción de Módulos y Servicios
13. Modelo de Datos
14. Integración con Inteligencia Artificial
15. Integración con Supabase y Autenticación
16. Flujo General del Sistema
17. Instalación y Configuración
18. Variables de Entorno
19. Pruebas y Validación
20. Ventajas del Sistema
21. Posibles Mejoras Futuras
22. Conclusiones

1. Introducción

SnitchAI es una plataforma web de generación automática de aplicaciones móviles basada en inteligencia artificial generativa. El sistema fue desarrollado utilizando tecnologías modernas como Next.js 16, TypeScript, Supabase y Google Generative AI (modelo Gemma 4).

La finalidad del proyecto es permitir que cualquier usuario pueda describir mediante lenguaje natural las características de una aplicación móvil y que el sistema genere automáticamente código funcional en React Native, reduciendo la barrera de entrada al desarrollo móvil.

El proyecto implementa una arquitectura modular y escalable basada en capas de servicios (Services), repositorios (Repositories) y enrutadores (Routers), facilitando el mantenimiento y la evolución futura del sistema. La persistencia de datos se centraliza en Supabase (PostgreSQL), mientras que la autenticación de usuarios es gestionada de forma nativa por el mismo proveedor.

2. Objetivos del Proyecto

Objetivo General

Desarrollar una plataforma inteligente capaz de generar aplicaciones móviles mediante inteligencia artificial a partir de descripciones en lenguaje natural realizadas por el usuario, automatizando el proceso de prototipado inicial de software móvil.

Objetivos Específicos

- Implementar un sistema de conversación contextual multi-turno con IA, manteniendo el historial completo de interacciones.
- Generar automáticamente código React Native (Expo) funcional y estructurado a partir de los requerimientos del usuario.
- Mantener y gestionar el historial de conversaciones de cada proyecto de forma persistente.
- Administrar versiones del código generado, permitiendo recuperar iteraciones previas.
- Detectar automáticamente requerimientos y pantallas a partir del análisis de la respuesta de la IA.
- Persistir toda la información del sistema utilizando Supabase como backend-as-a-service.

- Implementar autenticación de usuarios con soporte para credenciales propias, Google y GitHub.
- Garantizar la escalabilidad y mantenibilidad del sistema mediante una arquitectura en capas desacopladas.

3. Alcance del Sistema

Funcionalidades Incluidas

- Registro, inicio de sesión y gestión de sesiones de usuario (email/contraseña, Google, GitHub).
- Creación y administración de proyectos de aplicaciones móviles por usuario.
- Conversaciones iterativas entre el usuario y la IA con historial persistente.
- Generación de código React Native (Expo) completo y funcional.
- Detección y registro automático de requerimientos funcionales desde los prompts.
- Detección automática y almacenamiento de las pantallas de la app generada.
- Control de versiones del código generado en cada iteración.
- Persistencia completa de datos en base de datos relacional (PostgreSQL vía Supabase).
- Exportación de proyectos (módulo `export.service.ts` / `export.router.ts`).
- Vista previa de versiones (módulo `preview.repository.ts`).

Funcionalidades Excluidas en la Versión Actual

- Generación automática de APK (infraestructura preparada, implementación pendiente).
- Despliegue automático en tiendas de aplicaciones.
- Colaboración en tiempo real entre múltiples usuarios sobre el mismo proyecto.
- Editor visual de código integrado en la plataforma.

4. Descripción General del Problema

El desarrollo de aplicaciones móviles requiere tiempo, conocimientos técnicos especializados y experiencia en programación. Frameworks como React Native, aunque potentes, presentan una curva de aprendizaje considerable para personas sin formación en desarrollo de software.

Muchas personas y organizaciones poseen ideas válidas para aplicaciones móviles, pero carecen de los recursos técnicos o económicos para llevarlas a cabo. El prototipado tradicional es lento y costoso.

SnitchAI soluciona este problema aprovechando modelos de lenguaje de gran escala (LLM) para automatizar la generación del código inicial de aplicaciones móviles a partir de descripciones en lenguaje natural, democratizando el acceso al desarrollo móvil y acelerando significativamente el proceso de prototipado.

5. Requerimientos del Sistema

Requerimientos de Hardware

- Procesador de 64 bits, 2 núcleos o más recomendados.
- 8 GB de RAM recomendados (mínimo 4 GB).
- Espacio mínimo de almacenamiento: 2 GB para dependencias y proyecto.
- Conexión a internet (requerida para la API de Gemini y Supabase).

Requerimientos de Software

- Node.js 20 LTS o superior.
- npm 10 o superior (incluido con Node.js).
- Editor de código recomendado: Visual Studio Code.
- Cuenta activa en Supabase (plan gratuito suficiente para desarrollo).
- API Key de Google Generative AI (Gemini) con acceso al modelo gemma-4-it.
- Navegador web moderno (Chrome, Firefox, Edge).

6. Requerimientos Funcionales

ID	Requerimiento	Descripción Técnica	Criterio de Aceptación
RF-01	Autenticación de Usuarios	El sistema debe permitir registro e inicio de sesión mediante email/contraseña, Google y GitHub a través de Supabase Auth.	El usuario obtiene una sesión válida y es registrado en la tabla usuarios con su UUID de Supabase Auth.
RF-02	Gestión de Proyectos	El sistema debe permitir crear, listar, actualizar y eliminar proyectos de aplicaciones móviles asociados al usuario autenticado.	Cada proyecto genera un UUID único en la base de datos con el estado inicial "definido".

ID	Requerimiento	Descripción Técnica	Criterio de Aceptación
RF-03	Persistencia de Conversaciones	El sistema debe almacenar el historial completo de interacciones entre el usuario y la IA, vinculado a un proyecto.	Cada mensaje se guarda con proyecto_id, emisor (usuario/ia/sistema), tipo_mensaje y marca de tiempo.
RF-04	Procesamiento de Prompts Contextuales	El sistema debe inyectar el historial previo de la conversación al enviar un nuevo prompt a la API de Gemini.	La IA recibe el historial formateado correctamente como arreglo de turns (role/parts) sin perder el hilo conceptual.
RF-05	Generación de Código React Native	El sistema debe interpretar la respuesta estructurada de la IA y extraer el bloque de código fuente móvil.	El código extraído cumple con la sintaxis válida de React Native (Expo) y se persiste como ruta_codigo_fuente en la versión.
RF-06	Control de Versiones del Código	El sistema debe registrar un histórico de versiones del código generado en cada iteración del proyecto con numeración automática (v1.0, v2.0...).	El usuario puede consultar el código de cualquier versión pasada mediante su identificador.
RF-07	Detección Automática de Pantallas	El sistema debe analizar la respuesta de la IA para identificar y listar automáticamente las interfaces sugeridas para la aplicación.	Las pantallas detectadas se parsean a un arreglo de texto y se almacenan de forma independiente vinculadas a la versión.
RF-08	Extracción de Requerimientos	El sistema debe identificar los módulos o requerimientos implícitos en la descripción del usuario mediante el análisis de la IA.	Los requerimientos detectados (ej. "Autenticación", "CRUD") se indexan en la tabla requerimientos con tipo_requerimiento y mensaje_origen_id.
RF-09	Iteración de Proyectos	El sistema debe permitir al usuario enviar modificaciones sobre el código ya generado para refinarlo de forma iterativa.	La IA recibe el código de la versión actual junto con el nuevo prompt del usuario y genera una versión actualizada.
RF-10	Exportación de Proyectos	El sistema debe permitir exportar el código generado de un proyecto mediante el servicio de exportación.	El usuario puede descargar o exportar el código de la versión seleccionada a través del endpoint de exportación.

7. Requerimientos No Funcionales

ID	Requerimiento	Descripción	Criterio de Aceptación
RNF-01	Escalabilidad	La arquitectura debe soportar incremento de usuarios y módulos sin degradar la infraestructura principal.	Supabase como backend Serverless absorbe picos de tráfico concurrentes de forma automática.
RNF-02	Arquitectura Modular	El backend debe implementar separación estricta de responsabilidades (Separation of Concerns).	El código está dividido en capas independientes: Routers, Services y Repositories, sin dependencias cruzadas.
RNF-03	Eficiencia y Rendimiento	El sistema debe procesar e interactuar con la interfaz de forma fluida, optimizando los tiempos de cómputo locales.	Tiempo de respuesta de rutas internas (API Routes) inferior a 200 ms, excluyendo el tiempo de la API de Google.
RNF-04	Seguridad de Credenciales	El sistema no debe exponer llaves privadas, tokens ni credenciales de servidor en el lado cliente o repositorio público.	Las variables GEMINI_API_KEY y SUPABASE_SERVICE_ROLE_KEY se inyectan exclusivamente mediante variables de entorno en servidor (.env.local).
RNF-05	Mantenibilidad	El código debe ser autodescriptivo y estructurado de forma que cualquier desarrollador pueda integrar nuevas características rápidamente.	Cumplimiento de las reglas de estilo de ESLint con configuración estricta (eslint-config-next).
RNF-06	Tipado Estático	Toda la lógica de negocio, contratos de datos e interfaces deben estar tipados en TypeScript.	Cobertura del 100% del código en archivos .ts y .tsx, prohibiendo el uso del tipo any genérico.

8. Arquitectura del Sistema

SnitchAI implementa una arquitectura en capas desacopladas (Layered Architecture) sobre el framework Next.js 16 con App Router. Esta arquitectura garantiza la separación de responsabilidades y facilita el mantenimiento y la extensibilidad del sistema.

Capas Principales

Capa de Presentación (Frontend)

Gestiona la interfaz gráfica de usuario (GUI), la reactividad de los componentes, la captura de prompts del usuario y la renderización en tiempo real del código React Native generado.

Ubicación: /app/

Capa de Enrutamiento (Routers)

Actúa como pasarela de comunicación interna entre el cliente y el servidor. Implementa las API Routes nativas de Next.js para interceptar solicitudes HTTP, validar parámetros de entrada, gestionar códigos de estado HTTP y delegar el flujo hacia los servicios correspondientes.

Ubicación: /lib/routers/

Routers implementados: ChatRouter, ProyectosRouter, AuthRouter, VersionesRouter, ExportRouter, IARouter

Capa de Lógica de Negocio (Services)

Núcleo operativo del sistema donde se implementan las reglas de negocio principales. Coordina los repositorios y la integración con servicios externos (Gemini AI).

Ubicación: /lib/services/

Servicios implementados: ChatService, IAService, AuthService, ProyectosService, VersionesService, ExportService

Capa de Acceso a Datos (Repositories)

Implementa el patrón Repository como capa de abstracción sobre la base de datos. Aisla las consultas de lectura y escritura del resto de la aplicación, exponiendo métodos limpios y fuertemente tipados para que la lógica de negocio nunca dependa directamente de la sintaxis específica del motor de base de datos.

Ubicación: /lib/repositories/

Repositorios: ConversacionesRepository, MensajesRepository, IteracionesRepository, VersionesRepository, PantallasRepository, RequerimientosRepository, ProyectosRepository, UsuariosRepository, ApksRepository, PreviewsRepository

Capa de Infraestructura y Persistencia (Supabase)

Centraliza la inicialización, configuración y administración de las credenciales de acceso a Supabase. Provee tres clientes especializados: cliente administrativo (service role), cliente del servidor (SSR con cookies), y cliente del navegador (browser client).

Ubicación: /lib/supabase/

9. Tecnologías Utilizadas

Tecnología / Versión	Categoría	Función en el Sistema
Next.js 16.2.4	Framework Web	Framework principal, App Router, API Routes, SSR
React 19.2.4	Biblioteca UI	Construcción de la interfaz gráfica de usuario
TypeScript 5	Lenguaje	Tipado estático en todo el código fuente
Tailwind CSS 4	Estilos	Diseño y maquetado de la interfaz
Supabase 2.105	Backend-as-a-Service	Base de datos PostgreSQL, autenticación, storage
@supabase/ssr 0.10	Integración	Cliente SSR de Supabase para Next.js con cookies
@google/generative-ai 0.24	IA	SDK oficial de Google Generative AI (Gemini)
Modelo: gemma-4-it	Modelo de IA	Modelo de lenguaje utilizado para generación de código
React Native (Expo)	Target de Generación	Framework móvil para el código generado por la IA
ESLint 9	Calidad de Código	Análisis estático y cumplimiento de estándares

10. Diseño del Sistema

El sistema implementa separación estricta de responsabilidades mediante el patrón de arquitectura en capas. Cada módulo cumple una función específica y se comunica con las capas adyacentes a través de interfaces bien definidas:

- **Routers** → Validan la solicitud HTTP y delegan al Service correspondiente.
- **Services** → Implementan la lógica de negocio y coordinan los Repositories.
- **Repositories** → Encapsulan las operaciones de acceso a la base de datos.
- **Frontend** → Consume los endpoints definidos por los Routers.

Esta arquitectura facilita el mantenimiento independiente de cada capa, la escalabilidad horizontal, la reutilización de componentes y la integración futura de nuevas funcionalidades sin afectar el resto del sistema.

11. Estructura del Proyecto

```
AIMOBILE-main/
├─ app/                                # Capa de presentación (Next.js App Router)
│   ├── globals.css                    # Estilos globales (Tailwind CSS)
│   ├── layout.tsx                     # Layout raíz de la aplicación
│   └─ page.tsx                        # Página principal (punto de entrada visual)
├─ lib/                                # Lógica de negocio e infraestructura
│   ├── repositories/                 # Capa de acceso a datos
│   │   ├── conversaciones.repository.ts
│   │   ├── mensajes.repository.ts
│   │   ├── iteraciones.repository.ts
│   │   ├── versiones.repository.ts
│   │   ├── pantallas.repository.ts
│   │   ├── requerimientos.repository.ts
│   │   ├── proyectos.repository.ts
│   │   ├── usuarios.repository.ts
│   │   ├── apks.repository.ts
│   │   ├── previews.repository.ts
│   │   └─ index.ts
│   ├── routers/                      # Capa de enrutamiento (API Routes)
│   │   ├── auth.router.ts
│   │   ├── chat.router.ts
│   │   ├── ia.router.ts
│   │   ├── proyectos.router.ts
│   │   └─ versiones.router.ts
```

```
| | | └─ export.router.ts
| | └─ index.ts
| └─ services/                # Capa de lógica de negocio
| | └─ ia.service.ts
| | └─ chat.service.ts
| | └─ auth.service.ts
| | └─ proyectos.service.ts
| | └─ versiones.service.ts
| | └─ export.service.ts
| | └─ index.ts
| └─ supabase/                # Infraestructura y clientes Supabase
| | └─ admin.ts               # Cliente administrativo (service role)
| | └─ client.ts              # Cliente de navegador (browser client)
| | └─ server.ts              # Cliente de servidor (SSR + cookies)
| └─ types/
|   └─ database.types.ts      # Tipos TypeScript del modelo de datos
└─ public/                    # Assets estáticos
└─ package.json
└─ tsconfig.json
└─ eslint.config.mjs
└─ next.config.ts
```

12. Descripción de Módulos y Servicios

IAService (ia.service.ts)

Responsable de toda la comunicación con Google Generative AI. Encapsula la inicialización del modelo, la construcción del historial de conversación y el parseo de la respuesta estructurada de la IA.

Funciones principales:

- `chat(prompt, historial)`: Envía un mensaje con contexto completo al modelo Gemma 4 y retorna la respuesta parseada.
- `generarAppDesdePrompt(prompt)`: Generación directa sin historial previo.
- `parseResponse(text)`: Parsea la respuesta JSON de la IA extrayendo mensaje, código, pantallas y requerimientos.
- `buildHistory(mensajes)`: Transforma los mensajes de la BD al formato de historial requerido por la API de Gemini.

ChatService (chat.service.ts)

Orquestador principal del flujo conversacional. Coordina todos los repositorios para gestionar el ciclo completo de vida de un mensaje.

Funciones principales:

- `enviarMensaje(proyecto_id, contenido)`: Flujo completo — guarda mensaje del usuario, llama a IAService, guarda respuesta IA, registra requerimientos y versiones.
- `obtenerHistorial(proyecto_id)`: Recupera el historial ordenado de mensajes de un proyecto.
- `guardarVersion(proyecto_id, codigo, pantallas)`: Crea nueva iteración y versión, registra las pantallas detectadas.

AuthService (auth.service.ts)

Gestiona la autenticación y autorización de usuarios utilizando Supabase Auth. Este módulo no aparecía documentado en la versión original del documento.

Funciones principales:

- registrar(email, password, nombre): Crea cuenta en Supabase Auth y perfil en tabla usuarios.
- login(email, password): Inicio de sesión con credenciales propias.
- loginConGoogle() / loginConGithub(): Inicio de sesión OAuth con proveedores externos.
- logout(): Cierre de sesión.
- getUsuarioActual(): Recupera el perfil del usuario autenticado.
- recuperarPassword(email): Envía correo de recuperación de contraseña.

ProyectosService (proyectos.service.ts)

Gestiona el ciclo de vida completo de los proyectos: creación, listado, actualización, cambio de estado y eliminación.

VersionesService (versiones.service.ts)

Administra la recuperación y consulta de versiones de código generado para un proyecto.

ExportService (export.service.ts)

Gestiona la exportación del código generado de un proyecto. Permite al usuario obtener el código de una versión específica para usarlo fuera de la plataforma.

13. Modelo de Datos

El sistema trabaja con las siguientes entidades persistidas en PostgreSQL vía Supabase. Todas las entidades están tipadas en /lib/types/database.types.ts.

Entidad	Campos Clave	Descripción
Usuario	id (UUID), nombre, rol, estado, fecha_registro	Perfil del usuario en el sistema, vinculado al UUID de Supabase Auth.
Proyecto	id, usuario_id, nombre_proyecto, descripcion_inicial, objetivo, estado, presupuesto_estimado	Proyecto de aplicación móvil creado por el usuario. Estado: definido / en_desarrollo / en_pruebas / finalizado / cancelado.

Entidad	Campos Clave	Descripción
Conversacion	id, proyecto_id, titulo_conversacion, estado, fecha_ultima_actualizacion	Canal de comunicación IA-usuario asociado a un proyecto. Estado: activa / cerrada / archivada.
Mensaje	id, conversacion_id, emisor, contenido, tipo_mensaje, orden_mensaje	Cada interacción individual. Emisor: usuario / ia / sistema. Tipo: consulta / respuesta / sugerencia / error / sistema.
Requerimiento	id, proyecto_id, mensaje_origen_id, tipo_requerimiento, descripcion, prioridad, estado	Requerimiento detectado automáticamente por la IA. Tipo: funcional / no_funcional. Estado: pendiente / aprobado / implementado / descartado.
Iteracion	id, proyecto_id, numero_iteracion, objetivo_iteracion, estado, fecha_inicio, fecha_cierre	Ciclo de desarrollo que agrupa varias versiones. Estado: abierta / en_proceso / cerrada.
VersionAplicacion	id, proyecto_id, iteracion_id, numero_version, ruta_codigo_fuente, framework_objetivo, estado_generacion	Snapshot del código generado en una iteración. El código se almacena en ruta_codigo_fuente. Estado: generada / validada / rechazada / en_revision.
Pantalla	id, version_id, nombre_pantalla, tipo_pantalla, orden_visual, descripcion_funcional, estado	Pantalla detectada automáticamente en el código generado. Estado: activa / modificada / eliminada.
Preview	id, version_id, url_preview, storage_path, fecha_expiracion, estado	Vista previa generada de una versión. Estado: disponible / expirado / eliminado.
ApkGenerado	id, version_id, nombre_archivo, storage_path, version_code, version_name, tamano_mb, estado	APK generado para una versión (funcionalidad futura). Estado: generado / descargado / fallido.

14. Integración con Inteligencia Artificial

El sistema utiliza Google Generative AI a través del SDK oficial `@google/generative-ai` v0.24.1. El modelo configurado es `gemma-4-it`, inicializado al arranque del servicio.

Configuración del Modelo

```
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!)  
  
const MODEL = "gemma-4-it"
```

System Prompt

El modelo recibe un system prompt que lo configura como arquitecto de aplicaciones móviles. Las instrucciones clave son:

- Generar exclusivamente código React Native (Expo).
- Responder únicamente en formato JSON con la estructura: { mensaje, codigo, pantallas, requerimientos }.
- Extraer requerimientos y confirmar con el usuario antes de generar código en el primer prompt.
- Aplicar cambios incrementales cuando el usuario itera sobre código ya generado.

Estructura de Respuesta de la IA

```
{  
  "mensaje": "Respuesta conversacional al usuario",  
  "codigo": "// Código React Native completo (null si solo es conversación)",  
  "pantallas": ["Login", "Home", "Perfil"],  
  "requerimientos": ["CRUD", "Autenticación", "Notificaciones"]  
}
```

Manejo de Historial Multi-turno

El `IAService` construye el historial de conversación mapeando los mensajes de la base de datos al formato de turns requerido por la API de Gemini (role: "user" | "model", parts: [{text}]). El método `chat()` inyecta este historial en cada petición, garantizando la coherencia contextual de la conversación.

Parseo de Respuesta

El método `parseResponse()` limpia los bloques de código Markdown (``json) antes de hacer `JSON.parse()`. En caso de fallo en el parseo, retorna la respuesta como texto plano en el campo `mensaje`, evitando que un error de formato rompa el flujo del sistema.

15. Integración con Supabase y Autenticación

Supabase es utilizado como backend principal del sistema, proveyendo base de datos relacional (PostgreSQL), autenticación de usuarios, almacenamiento y funciones serverless.

Cientes de Supabase

Archivo	Cliente	Uso
/lib/supabase/admin.ts	<code>createClient</code> (service role)	Operaciones administrativas del lado del servidor. Usa <code>SUPABASE_SERVICE_ROLE_KEY</code> . Nunca expuesto al cliente.
/lib/supabase/server.ts	<code>createServerClient</code> (SSR)	Operaciones del servidor en Next.js con gestión de cookies. Usa <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> . Soporta autenticación basada en sesiones.
/lib/supabase/client.ts	<code>createBrowserClient</code>	Operaciones desde el navegador (componentes cliente). Usa <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> .

Autenticación

El `AuthService` implementa autenticación completa a través de Supabase Auth con soporte para:

- Email y contraseña (registro y login).
- OAuth con Google (`signInWithOAuth` provider: "google").
- OAuth con GitHub (`signInWithOAuth` provider: "github").
- Recuperación de contraseña vía email con redirección a `/reset-password`.

Al registrarse, el UUID generado por Supabase Auth se utiliza directamente como clave primaria del perfil de usuario en la tabla `usuarios`, garantizando la consistencia entre el sistema de autenticación y el modelo de datos.

Variables de Entorno Requeridas

Variable	Entorno	Descripción
GEMINI_API_KEY	Servidor	Clave de API de Google Generative AI. No exponer al cliente.
NEXT_PUBLIC_SUPABASE_URL	Público	URL del proyecto Supabase. Accesible desde el cliente y el servidor.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Público	Clave anónima de Supabase. Accesible desde el cliente con RLS activo.
SUPABASE_SERVICE_ROLE_KEY	Servidor	Clave de rol de servicio. Solo en servidor. Acceso completo sin RLS.
NEXT_PUBLIC_APP_URL	Público	URL base de la aplicación. Usada para redirección en recuperación de contraseña.

16. Flujo General del Sistema

Flujo de Autenticación

1. El usuario accede a la plataforma y se registra o inicia sesión.
2. Supabase Auth valida las credenciales y emite una sesión.
3. El AuthService registra el perfil del usuario en la tabla usuarios.

Flujo de Generación de Aplicación

4. El usuario crea un nuevo proyecto (ProyectosService → ProyectosRepository).
5. El usuario escribe un prompt describiendo su aplicación.
6. ChatRouter.enviarMensaje() recibe la solicitud y la delega a ChatService.
7. ChatService persiste el mensaje del usuario en la base de datos.
8. ChatService recupera el historial completo de la conversación.
9. IAService.chat() envía el historial + nuevo prompt al modelo Gemma 4.
10. Gemini procesa la información y retorna una respuesta JSON estructurada.
11. ChatService persiste: mensaje de IA, requerimientos detectados, nueva versión del código y pantallas.
12. El frontend recibe la respuesta y renderiza el código generado al usuario.
13. El usuario puede continuar iterando con nuevos prompts de refinamiento.

17. Instalación y Configuración

Paso 1: Clonar el Repositorio

```
git clone https://github.com/ErickHub192/AIMOBILE.git
cd AIMOBILE
```

Paso 2: Instalar Dependencias

```
npm install
```

Paso 3: Configurar Variables de Entorno

Crear el archivo `.env.local` en la raíz del proyecto con el siguiente contenido:

```
GEMINI_API_KEY=tu_api_key_de_google_generative_ai
NEXT_PUBLIC_SUPABASE_URL=https://tu-proyecto.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=tu_clave_anonima_supabase
SUPABASE_SERVICE_ROLE_KEY=tu_clave_service_role_supabase
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Paso 4: Configurar la Base de Datos en Supabase

- Crear un nuevo proyecto en supabase.com.
- Ejecutar el script SQL de creación de tablas (ver sección de Modelo de Datos).
- Habilitar los proveedores OAuth deseados (Google, GitHub) en Authentication > Providers.
- Configurar las URLs de redirección en Authentication > URL Configuration.

Paso 5: Ejecutar el Proyecto

```
npm run dev
```

La aplicación estará disponible en `http://localhost:3000`.

18. Variables de Entorno

Ver sección 15 para la tabla completa de variables de entorno con descripción detallada y contexto de uso. A continuación se presenta el resumen ejecutivo:

Variable	Visibilidad	Propósito
GEMINI_API_KEY	Servidor	Acceso a Google Generative AI (Gemma 4)
NEXT_PUBLIC_SUPABASE_URL	Público	URL del proyecto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Público	Clave anónima con RLS activo
SUPABASE_SERVICE_ROLE_KEY	Servidor	Clave administrativa sin RLS
NEXT_PUBLIC_APP_URL	Público	URL base para redirecciones OAuth

19. Pruebas y Validación

El sistema puede validarse de forma manual y funcional mediante los siguientes escenarios de prueba:

Pruebas de Autenticación

- Registro de nuevo usuario con email y contraseña: verificar creación en tabla usuarios.
- Inicio de sesión con Google y GitHub: verificar generación de sesión válida.
- Recuperación de contraseña: verificar recepción de email con enlace válido.

Pruebas de Gestión de Proyectos

- Creación de proyecto: verificar UUID generado y estado inicial "definido".
- Listado de proyectos del usuario autenticado.
- Actualización de nombre y descripción.
- Cambio de estado del proyecto a lo largo del ciclo de vida.

Pruebas de Generación con IA

- Enviar prompt inicial: verificar que la IA solicita confirmación de requerimientos.
- Aprobar requerimientos: verificar generación de código React Native válido (v1.0).
- Verificar persistencia en tabla versiones con ruta_codigo_fuente poblada.

- Verificar detección y almacenamiento de pantallas en tabla pantallas.
- Verificar indexación de requerimientos en tabla requerimientos.

Pruebas de Iteración

- Enviar prompt de modificación sobre código ya generado.
- Verificar creación de versión v2.0 con el código actualizado.
- Verificar recuperación de versión v1.0 sin modificaciones.

Pruebas de Persistencia

- Verificar que el historial de mensajes se recupera correctamente tras recargar la página.
- Verificar el orden correcto de mensajes mediante orden_mensaje.

20. Ventajas del Sistema

Automatización del Prototipado

SnitchAI reduce drásticamente el tiempo necesario para crear un prototipo funcional de aplicación móvil, pasando de semanas a minutos en la generación inicial del código base.

Accesibilidad

Permite a usuarios sin conocimientos de programación participar activamente en la definición y generación de su aplicación mediante lenguaje natural.

Arquitectura Escalable

La separación en capas (Routers, Services, Repositories) y el uso de Supabase como backend serverless permite escalar la plataforma sin cambios estructurales significativos.

Trazabilidad Completa

El sistema mantiene un registro histórico completo de conversaciones, requerimientos detectados, versiones de código y pantallas generadas, ofreciendo total trazabilidad del proceso de desarrollo.

Seguridad

La separación entre cliente público (anon key + RLS) y servidor administrativo (service role), combinada con el manejo estricto de variables de entorno, garantiza la protección de las credenciales del sistema.

Tipado Estricto

El uso de TypeScript en todo el código fuente, junto con las interfaces definidas en `database.types.ts`, minimiza errores en tiempo de ejecución y mejora la experiencia de desarrollo.

21. Posibles Mejoras Futuras

- Generación automática de APK: La infraestructura (tabla `apks_generados`, `ApksRepository`) ya está preparada; falta implementar el proceso de compilación en servidor.
- Exportación mejorada de proyectos: Descarga directa del proyecto React Native como archivo ZIP listo para usar con Expo.
- Dashboard administrativo: Panel para visualizar métricas de uso, proyectos activos, versiones generadas y actividad de la IA.
- Sistema multiusuario y colaborativo: Permitir que varios usuarios trabajen sobre el mismo proyecto en tiempo real.
- Editor visual de código integrado: Visor y editor de código en el navegador con resaltado de sintaxis.
- Testing automático del código generado: Integración con Jest/Detox para validar automáticamente el código React Native generado.
- Previzualización en tiempo real: Renderizado interactivo del código generado en la plataforma (Snack SDK o similar).
- Soporte multi-modelo: Permitir al usuario elegir entre distintos modelos de IA (Gemini, GPT-4, Claude) según sus preferencias.
- Notificaciones: Sistema de alertas cuando la generación de código o APK finaliza.

22. Conclusiones

SnitchAI representa una solución moderna y funcional orientada a la democratización del desarrollo móvil mediante inteligencia artificial generativa. El sistema demuestra cómo la combinación estratégica de tecnologías contemporáneas puede crear valor tangible, reduciendo significativamente las barreras técnicas para la creación de aplicaciones móviles.

La arquitectura en capas implementada (Routers → Services → Repositories) garantiza un código base mantenible, extensible y comprensible por cualquier desarrollador que se integre al proyecto. La elección de Next.js 16 con App Router, TypeScript y Supabase como stack principal refleja un criterio técnico sólido y alineado con las mejores prácticas de la industria.

La integración con Google Generative AI (modelo Gemma 4) posibilita conversaciones multi-turno contextuales, generación de código React Native funcional, detección automática de requerimientos y pantallas, todo en una sola interacción natural con el usuario.

El sistema cuenta con un modelo de datos robusto y completo que soporta trazabilidad total del proceso (conversaciones, versiones, iteraciones, requerimientos, pantallas), autenticación multi-proveedor (email, Google, GitHub) y una base de infraestructura preparada para funcionalidades futuras como la generación de APK.

En conclusión, SnitchAI constituye un aporte significativo que evidencia cómo la inteligencia artificial puede integrarse de forma estructurada y profesional en el ciclo de desarrollo de software, acelerando el prototipado y poniendo al alcance de cualquier usuario la capacidad de crear aplicaciones móviles funcionales.
